CSC-239
Java Programming:
Chapter 6: Methods

September 22, 2025

NOTICE to Students: Portions of these lecture slides are
Copyright 2021 Pearson Education, Inc.
We have a license to use this material ***only*** in the context of this course.
There is NO PERMISSION to share these lecture slides with anyone not taking the course.
There is NO PERMISSION to post these lecture slides on the Internet.

Reminder

- Quiz 1 began on Monday, September 15.
  - Online (Moodle) Quiz.
  - The Quiz will remain open until Monday September 22 , at 11:00 p.m.
  - Each student will have two hours to complete the Quiz, from the time that they begin.
- The material covered on Quiz 1 will be
  - The content of the course Syllabus
  - The lectures of September 3 and 8.
  - The quiz will include True / False, and Multiple-Choice questions.

Reminder: Submitting Files to *Moodle*

- When you submit your source code (*.java) files to *Moodle*, you MUST follow the **Submit Assignment to *Moodle*** procedure *completely*: be sure that the final Submission Status is:

  "Submitted for grading".



- This procedure is described in detail in the **Submit_Assignment_to_Moodle.pdf** document, which is available on *Moodle*, under the "**Handouts**" topic.

Online Information

- Java Tutorials:

  https://docs.oracle.com/javase/tutorial/

- Java API:

  https://docs.oracle.com/en/java/javase/21/docs/api/index.html

- The official "*Grader Than*" documentation:

  https://docs.graderthan.com/student/

- *Grader Than* Support E-mail:

  support@graderthan.com

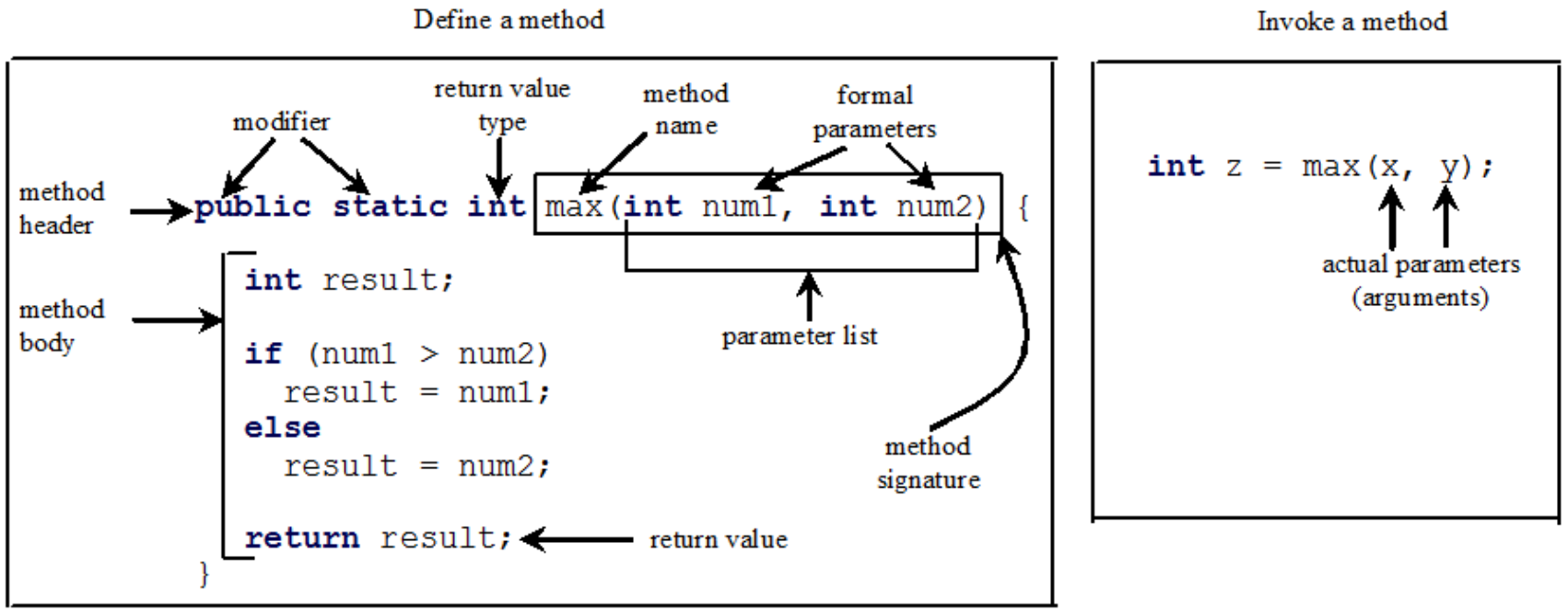
Why use Methods?

- Methods allow the re-use of portions of your program, to reduce redundant coding.
- Methods simplify program design by helping you divide a complex problem into multiple simpler problems. A modular design is easier to implement, verify, and enhance.
- **Question**: What's the difference between a C++ "*function*" and a Java "*method*"?

  Answer: not much. Most of the time, you can think of them of two different names for the same concept.

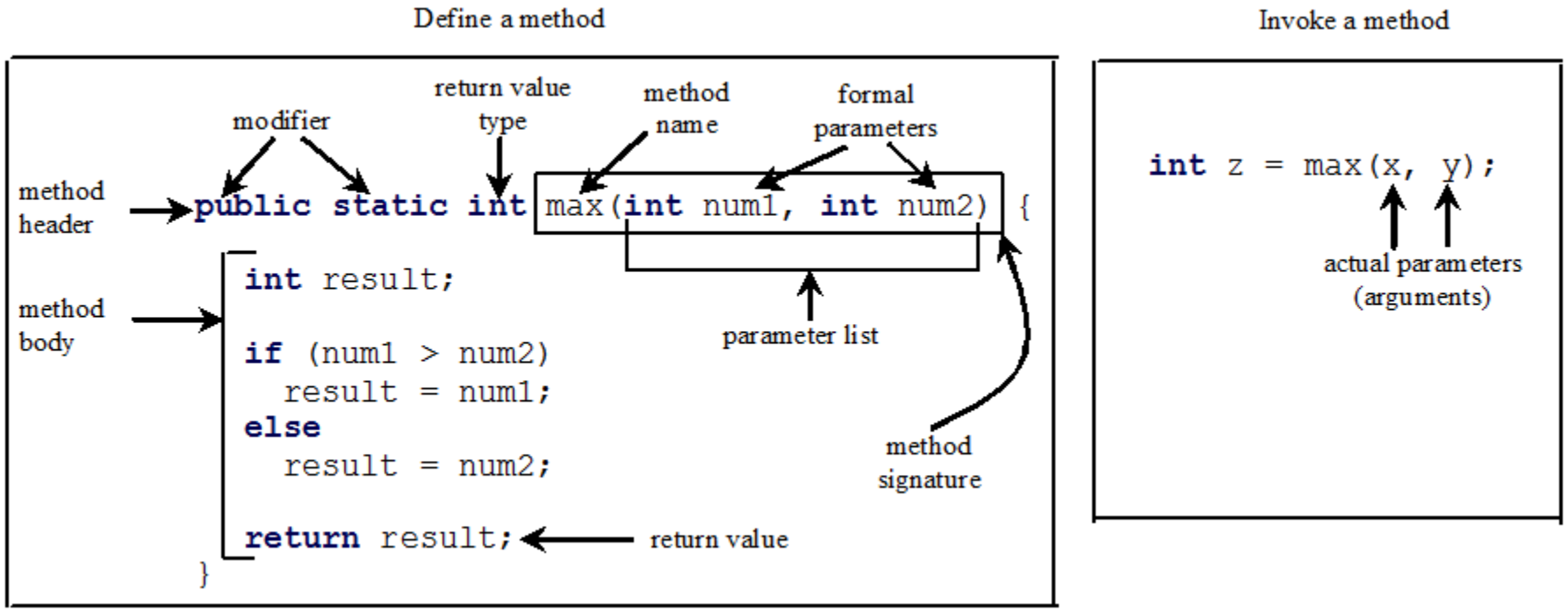
Defining Methods

A *method* is a collection of statements that are grouped together to perform some operation.



Method Signature (1)

- *Method signature* is the combination of the method name and the type, number, and order of the parameters.

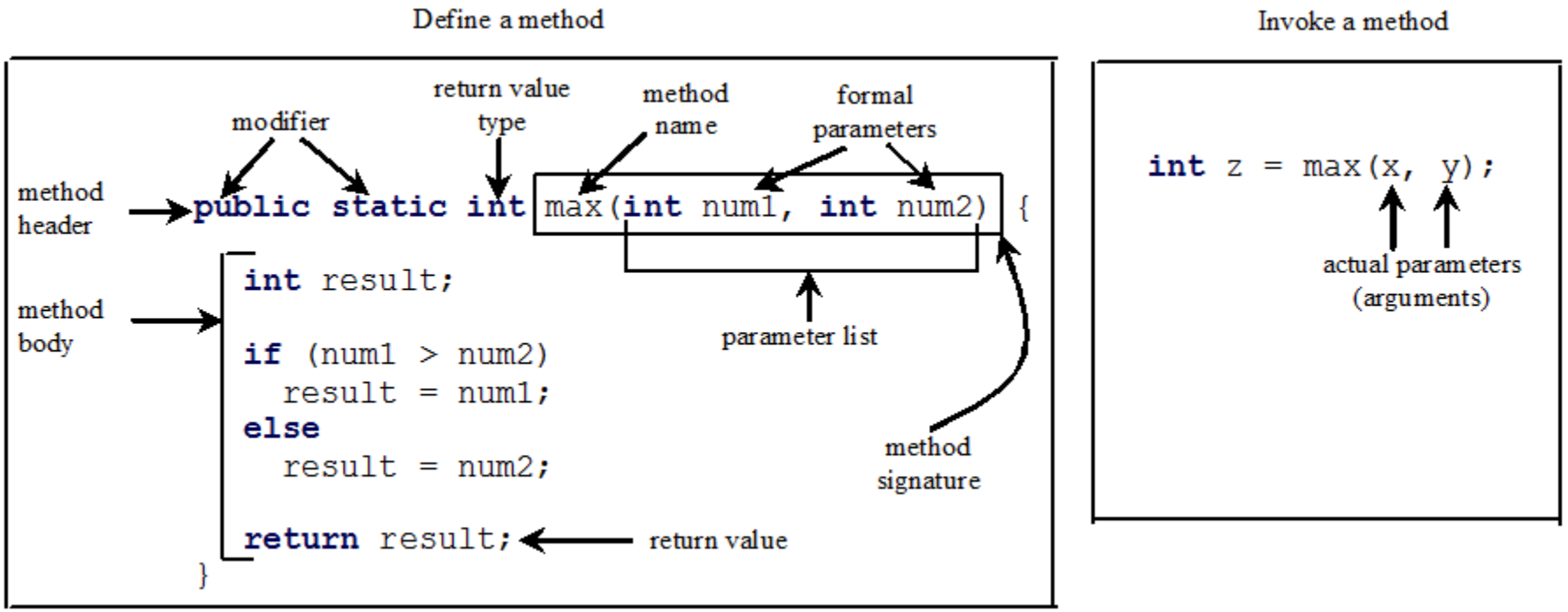


Method Signature (2)

- The method signature does *not* include the parameter names (only the type, number, and order).
- The method signature does *not* include the method return type.
- The idea of a method *signature* becomes important when we talk about "method overloading".

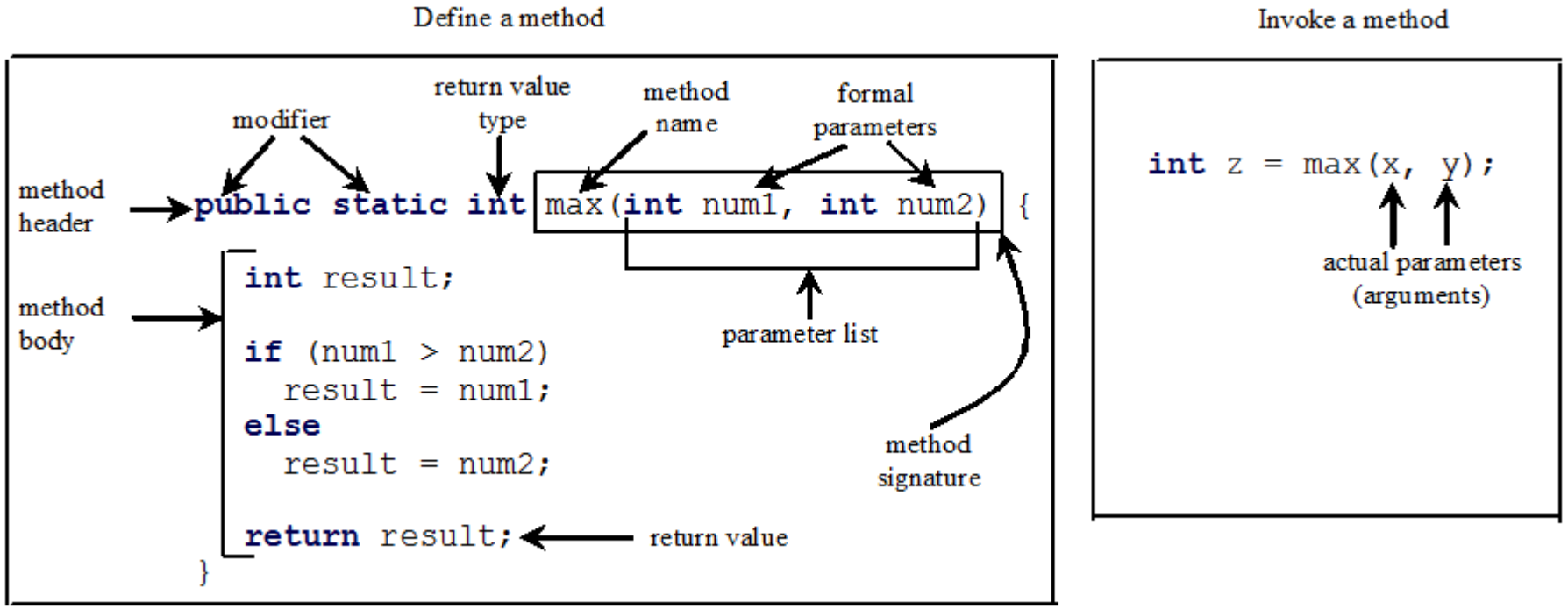
Formal Parameters

The variables defined in the method header are known as *formal parameters*.



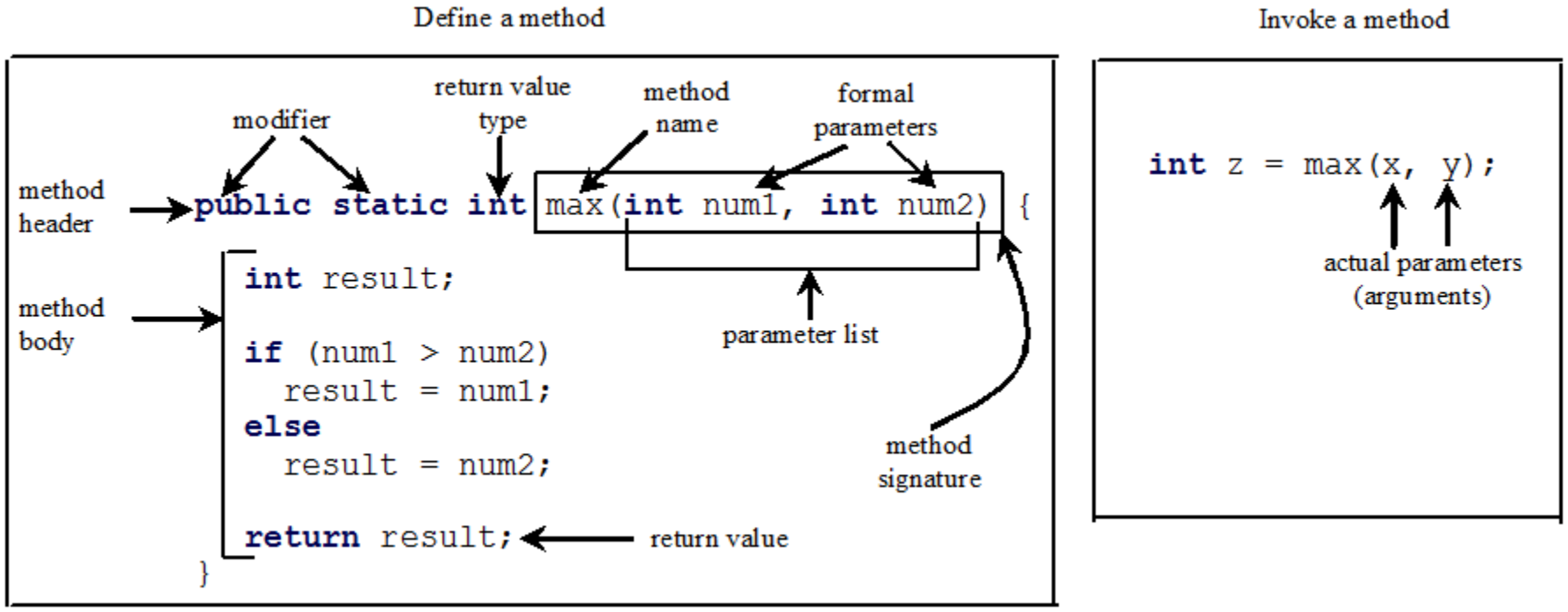
Actual Parameters

When a program "calls" (invokes) a method, it passes a value to the parameter. This value is referred to as ***actual parameter*** or ***argument***.



Return Value Type

A method may return a value. The return value type is the data type of the value the method returns. If the method does not return a value, the return value type is the keyword **void**. For example, the return value type in the main method is **void**.



CAUTION

A return statement is required for a value-returning method. The method shown below in (a) is logically correct, but it has a compilation error because the Java compiler "thinks" it might be possible for this method not to return any value.

```
public static int sign(int n) {
  if (n > 0)
    return 1;
  else if (n == 0)
    return 0;
  else if (n < 0)
    return -1;
}
```

(a)

Should be

```
public static int sign(int n) {
  if (n > 0)
    return 1;
  else if (n == 0)
    return 0;
  else
    return -1;
}
```

(b)

To fix this problem, remove *if (n < 0)* in (a), so that the compiler will see a return statement to be reached regardless of how the if statement is evaluated.

Executing (calling) a Method

- When we want our program to execute ("call") a particular method, our source code must use the method name, and whatever arguments the method requires (if any).
- The computer executes the method, and then resumes execution where it left off in the calling code.
  - If the method returned some value, then that value is used in whatever expression called the method.
- But how does it know where it came from?

  The "big idea" of code re-use is that your program can use a particular method at many locations in your program.

  How does the computer find its way back to the correct location?

The Call Stack

- In the process of "calling" a method, the computer saves the memory address of the **next** operation (after the method call) in a special area of memory that people refer to as the "call stack" or the "runtime stack". Then, when the method is ready to "return to its caller", the computer looks in the call stack for the return address.
- Actually, other information (in addition to the return address) is also placed on the call stack: any "local variables" defined inside the method exist on the call stack.
- In computer science, when we refer to a data structure as a "stack", what we generally mean is that the data is managed in a "Last-In, First-Out" (LIFO) manner.

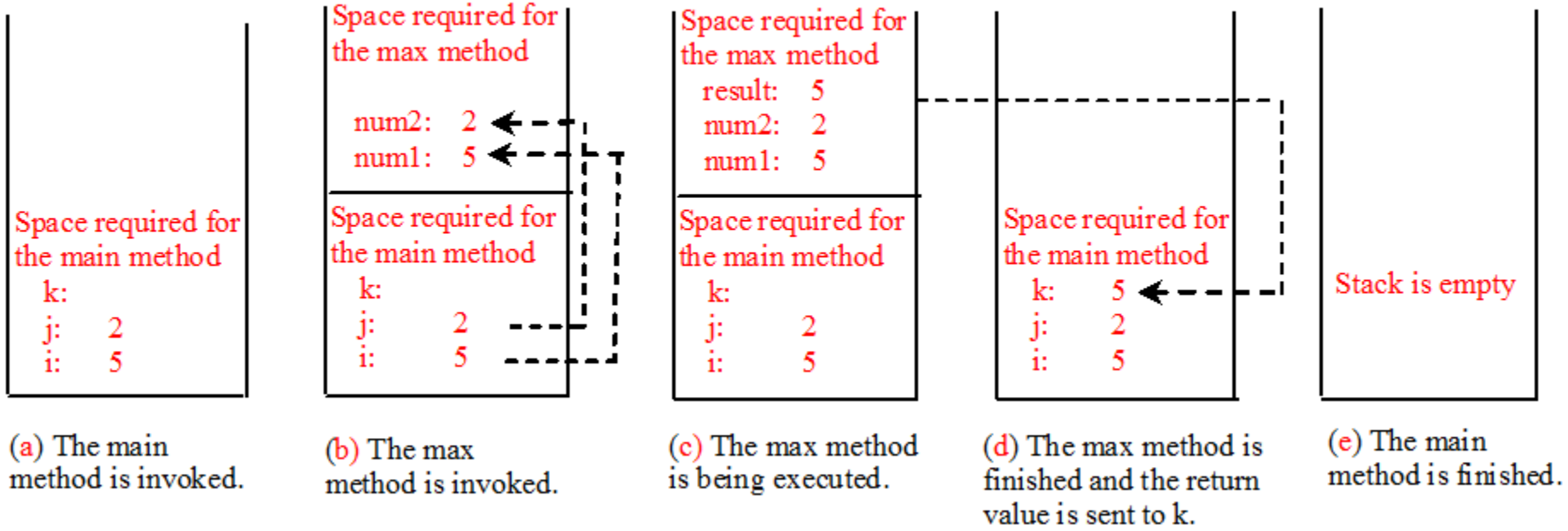
Demonstrate Call Stacks

```
public class TestMax {
   public static void main(String[] args) {
     int i = 5, j = 2;
     int k = max(i, j);
     System.out.println("The maximum between "
         + i + " and " + j + " is " + k);
   } // end main

/** Return the max between two numbers */
   public static int max(int num1, int num2) {
     int result;
     if (num1 > num2)
         result = num1;
     else
         result = num2;
     return result;
   } // end max
} // end TestMax
```

Call Stacks

- The diagram below illustrates (step-by-step) how data is present on the Call Stack



Demonstrate Passing Parameters

```
public class TestSwap {

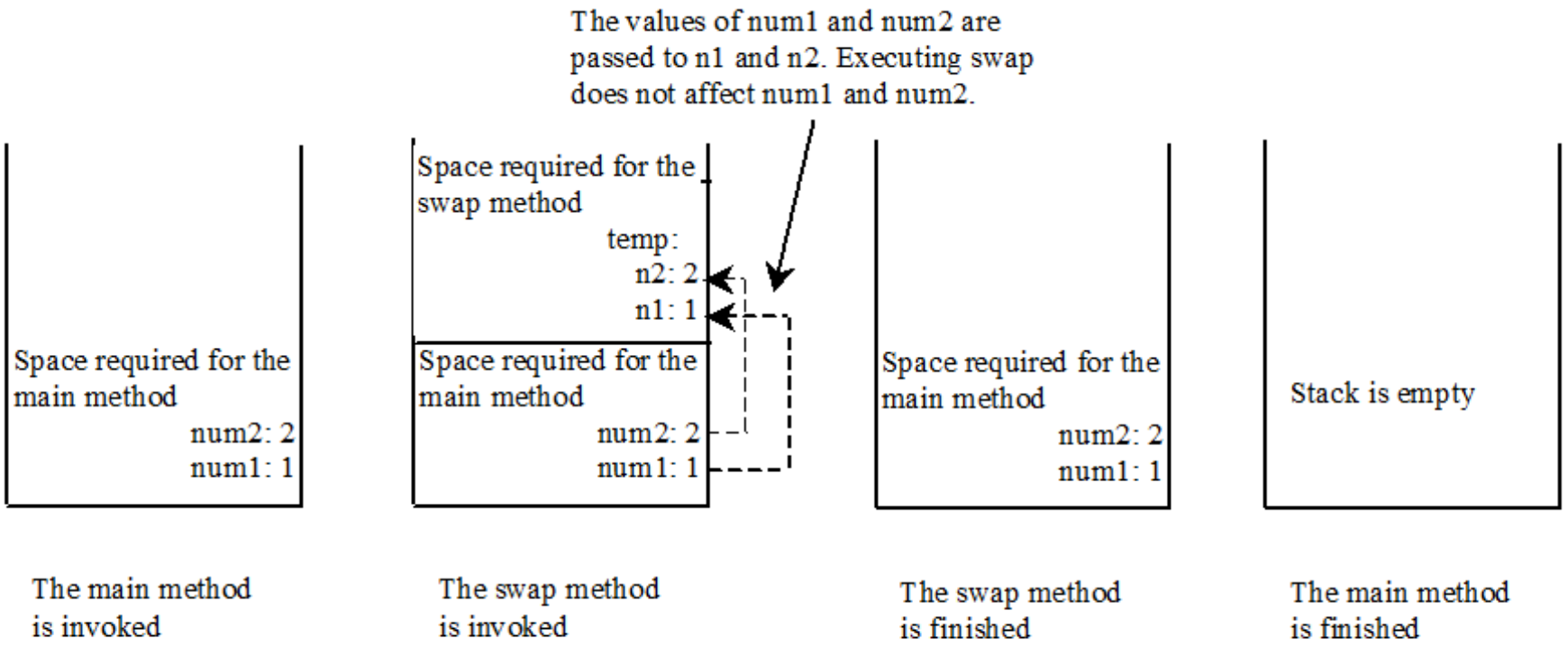

public static void main(String[] args) {
    int num1 = 1;
    int num2 = 2;
    swap(num1, num2);
    System.out.printf("num1=%d, num2=%d\n",
                                num1, num2);
  }

public static void swap(int n1, int n2) {
    int temp;
    temp = n1;
    n1 = n2;
    n2 = temp;
    System.out.printf("n1=%d, n2=%d\n", n1, n2);
  }
}
```

Output from `TestSwap`

```
Before: num1=1, num2=2
Inside: n1=2, n2=1
After: num1=1, num2=2
```

Passing Parameters by Value



Overloading Methods

- The version of `max()` shown below takes floating point parameters, rather than integers.

```
public static double max(double num1,
                         double num2) {
      if (num1 > num2)
         return num1;
      else
         return num2;
}
```

- The compiler will do its best to select the correct version of the method, based on the types of the arguments, even promoting types if necessary.

Ambiguous Invocation

Sometimes there may be two or more possible matches for an invocation of a method, but the compiler cannot determine the most specific match. This is referred to as *ambiguous invocation*. Ambiguous invocation is a compilation error.

Ambiguous Invocation Example

```
public class AmbiguousOverloading {
  public static void main(String[] args) {
    System.out.println(max(1, 2));
  }
  public static double max(int num1, double num2) {
    if (num1 > num2)
      return num1;
    else
      return num2;
  }
  public static double max(double num1, int num2) {
    if (num1 > num2)
      return num1;
    else
      return num2;
  }
}
```

Output from AmbiguousOverloading

```
java: reference to max is ambiguous
  both method max(int,double)
          in AmbiguousOverloading
  and method max(double,int)
          in AmbiguousOverloading match
```

Scope of Variables

- A **local variable**: a variable defined inside a method.
- **Scope**: the part of the program where the variable can be referenced.

- The scope of a local variable starts from its declaration and continues to the end of the block that contains the variable. A local variable must be declared before it can be used.

Nested vs. Non-nested Blocks

It is fine to declare i in two non-nesting blocks

```
public static void method1() {
  int x = 1;
  int y = 1;

  for (int i = 1; i < 10; i++) {
    x += i;
  }

  for (int i = 1; i < 10; i++) {
    y += i;
  }
}
```

It is wrong to declare i in two nesting blocks

```
public static void method2() {

  int i = 1;
  int sum = 0;

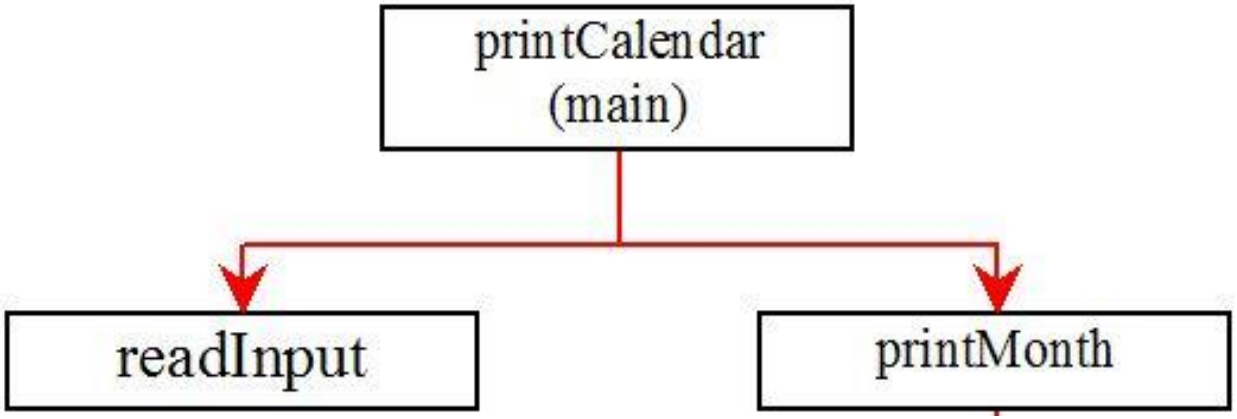
  for (int i = 1; i < 10; i++)
    sum += i;
  }
}
```

Stepwise Refinement

- When writing a large program, you can use the "divide and conquer" strategy, also known as *stepwise refinement*.
- Decompose the overall problem into sub-problems.
- Each sub-problem is (hopefully) easier to solve that the main problem.
- The sub-problems can be further decomposed into smaller, more manageable problems.

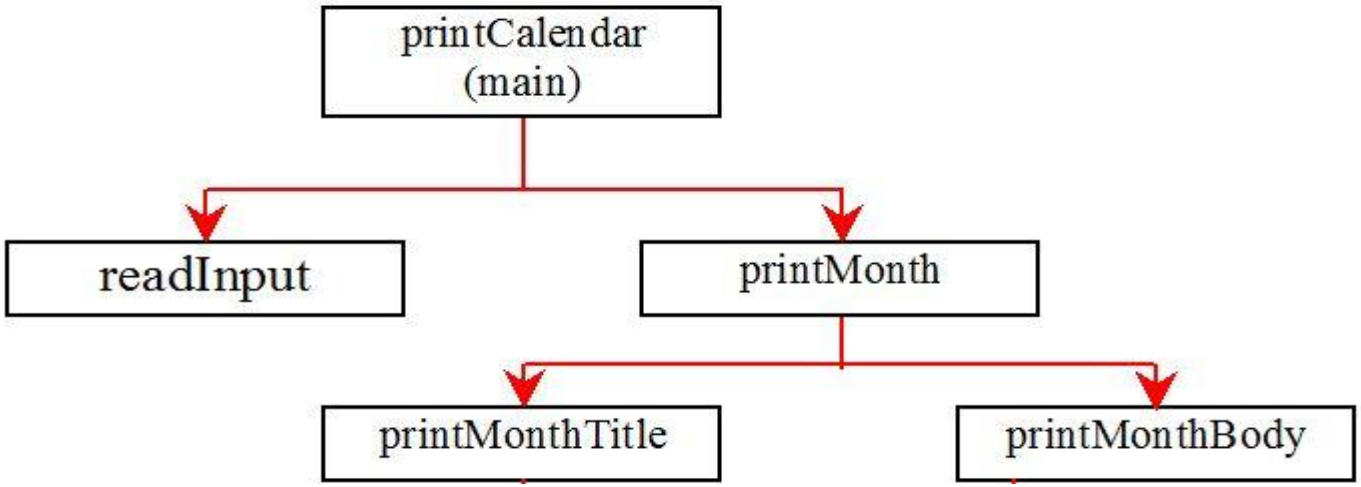
Example: Design Diagram

- Section 6.11 of the textbook (pages 227-234) discusses an example of designing and implementing a program to print a page for a calendar.
- Start with a simple problem statement (top level).
- Break the task into multiple (simpler) tasks (lower level of the diagram).

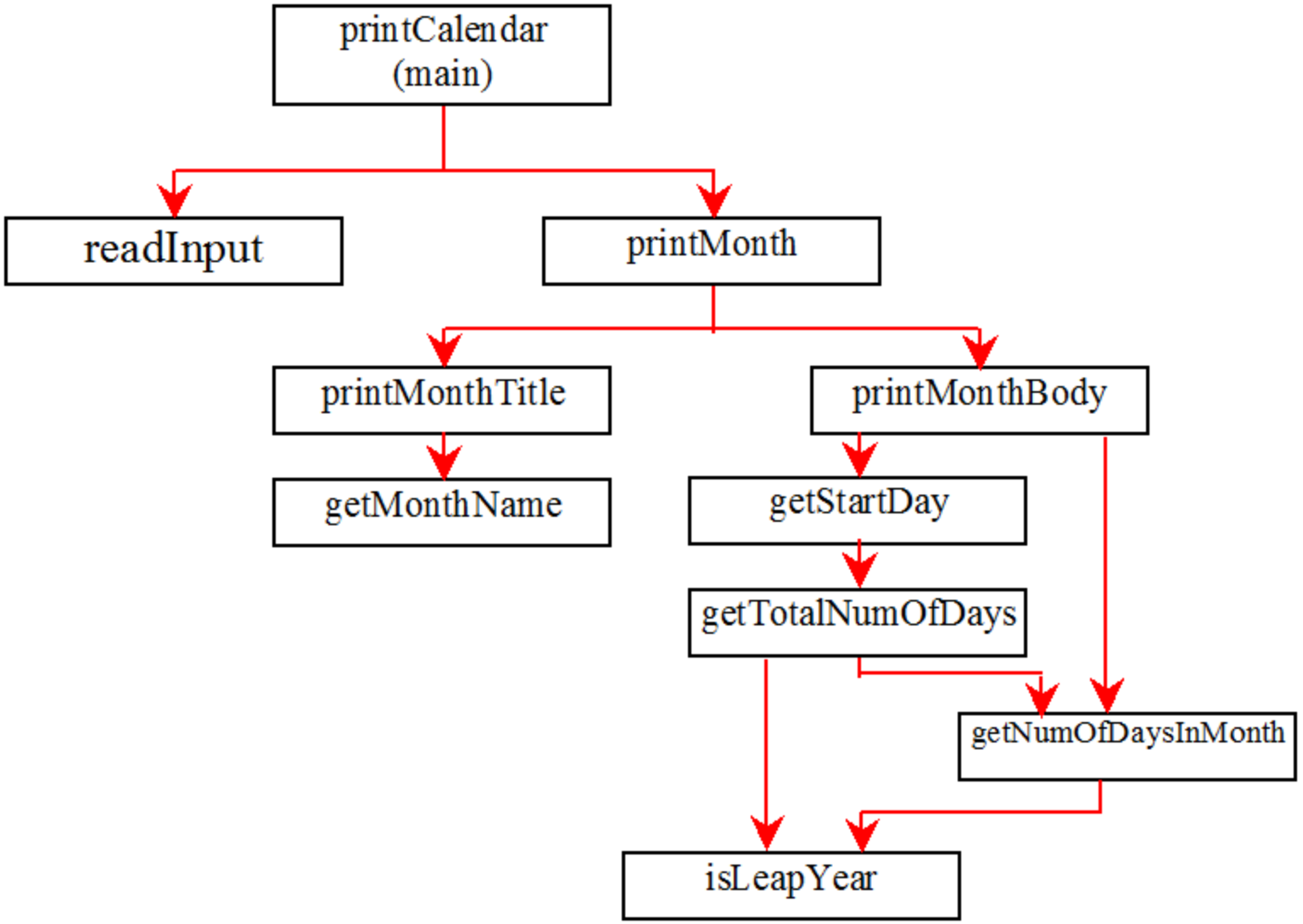


Continue to Add More Detail

- If a task seems complex, break it down into subtasks.
- Each box in the diagram will correspond to one method in the final code.



Eventually: Enough Detail



Approaches to Implementation

- Top-Down Implementation: code the upper-level methods first.
- Bottom-Up Implementation: code the lower-level methods first.

Implementation: Top-Down

- Implement methods from top level of chart first.
- Use “stubs” for methods which have not been coded yet.
- A “stub” implements the calling interface for the method, but does not do any meaningful work.

Implementation: Bottom-Up

- Implement methods from lower-level methods first.
- Need to write a “test driver” program to test the method (because its real caller does not exist yet).

Mixing "Top-Down" and "Bottom-Up"

- Often, the best implementation order is a "mix" of top-down and bottom-up approaches.
- Both are "incremental" approaches:
  - Code a little.
  - Debug what has been coded so far.
  - Code some more.
  - Debug the new code.
- Incremental development helps isolate errors, and makes debugging easier.
- As with any skill, deciding which parts to do first becomes easier as you gain more experience.

Project 1: Day of the Year

- Project requires you to write a program that inputs a **String** that represents a date in a particular year. For example:

```
03 05 2022
```

- The program must break up this string into its corresponding integer values of **Month**, **Day**, and **Year**, and then calculate **how many** days have elapsed from January 1 of that year to the specified date.
- Project 1 is due on Wednesday, October 15.